一、背景

spend-beforehand多次发生oom被容器kill

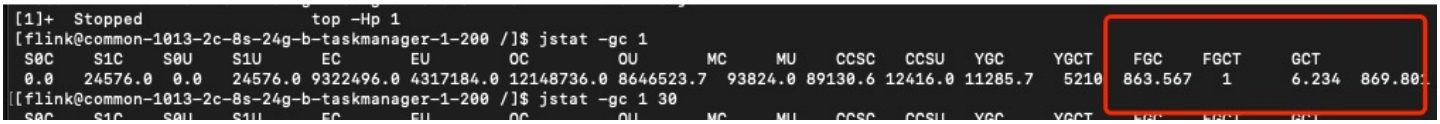
通过内存dashboard可以发现一些TM在内存缓慢积压到一定时刻，因为内存泄漏被容器kill，导致作业重启

二、查找问题

gc日志

一般出现内存泄露，首先需要查看，发现gc日志一切正常，job运行期间仅发生一次fullGC对应日志如下

jstat -gc 1



heap堆

此时查看了该TM的内存配置，此时老年代数据已占80%

jmap -heap 1

```
using thread-local object allocation.
Garbage-First (G1) GC with 2 thread(s)
```

```
   MaxHeapSize              = 22011707392 (20992.0MB)
   NewSize                  = 1363144 (1.2999954223632812MB)
   MaxNewSize               = 13203668992 (12592.0MB)
   OldSize                  = 5452592 (5.1999969482421875MB)
   NewRatio                 = 2
   SurvivorRatio            = 8
   MetaspaceSize            = 21807104 (20.796875MB)
   CompressedClassSpaceSize = 260046848 (248.0MB)
   MaxMetaspaceSize         = 268435456 (256.0MB)
   G1HeapRegionSize         = 8388608 (8.0MB)

Heap Usage:
G1 Heap:
   regions  = 2624
   capacity = 22011707392 (20992.0MB)
   used     = 13888062448 (13244.688461303711MB)
   free     = 8123644944 (7747.311538696289MB)
   63.093980856058074% used
G1 Young Generation:
Eden Space:
   regions  = 573
   capacity = 10720641024 (10224.0MB)
   used     = 4806672384 (4584.0MB)
   free     = 5913968640 (5640.0MB)
   44.83568075117371% used
Survivor Space:
   regions  = 5
   capacity = 41943040 (40.0MB)
   used     = 41943040 (40.0MB)
   free     = 0 (0.0MB)
   100.0% used
G1 Old Generation:
   regions  = 1139
   capacity = 11249123328 (10728.0MB)
   used     = 9031058416 (8612.688461303711MB)
   free     = 2218064912 (2115.311538696289MB)
   80.282330921921124% used

41309 interned Strings occupying 14318112 bytes.
```

heap堆中对象实例

此时使用jmap -histo:live 1命令查看内存中主要对象，可以看见guava Cache 的实例占用了很多内存空间，一共有5737451个localCache的存储键值对。

```
[flink@common-1013-2c-8s-24g-b-taskmanager-1-200 /]$ jmap -histo:live 1

 num     #instances         #bytes  class name
----------------------------------------------
   1:       5737451      367196864  org.apache.flink.shaded.guava18.com.google.common.cache.LocalCache$StrongAccessWriteEntry
   2:         21206      349885376  [B
   3:       6494059      259762360  com.mediav.overstep.spend.control.EstimateSpendDimension
   4:       4062962      162518480  java.math.BigDecimal
   5:       5737451      137698824  com.mediav.spend.beforehand.operator.EstimateSpendValueWithTime
   6:       4062939       97510536  scala.math.BigDecimal
   7:       5737451       91799216  org.apache.flink.shaded.guava18.com.google.common.cache.LocalCache$StrongValueReference
   8:        128740       77207184  [Ljava.lang.Object;
   9:        291731       28441776  [C
  10:        657299       26291960  java.math.BigInteger
  11:        892555       21421320  scala.collection.mutable.DefaultEntry
  12:        892032       21408768  com.mediav.spend.beforehand.domain.DataIndex
  13:        663253       16473808  [I
  14:        277166        8869312  java.util.HashMap$Node
  15:           351        7401328  [Lscala.collection.mutable.HashEntry;
```

fullGC

在等了一小段时间再次打印jmap -heap 1信息，可以推断此时发生了一次fullGC，通过查看GC信息发现确实进行了一次fullGC

```
[flink@common-1013-2c-8s-24g-b-taskmanager-1-200 /]$ jmap -heap 1
Attaching to process ID 1, please wait...
Debugger attached successfully.
```

```
Garbage-First (G1) GC with 2 thread(s)

Heap Configuration:
   MinHeapFreeRatio         = 40
   MaxHeapFreeRatio         = 70
   MaxHeapSize              = 22011707392 (20992.0MB)
   NewSize                  = 1363144 (1.2999954223632812MB)
   MaxNewSize               = 13203668992 (12592.0MB)
   OldSize                  = 5452592 (5.1999969482421875MB)
   NewRatio                 = 2
   SurvivorRatio            = 8
   MetaspaceSize            = 21807104 (20.796875MB)
   CompressedClassSpaceSize = 260046848 (248.0MB)
   MaxMetaspaceSize         = 268435456 (256.0MB)
   G1HeapRegionSize         = 8388608 (8.0MB)

Heap Usage:
G1 Heap:
   regions  = 2624
   capacity = 22011707392 (20992.0MB)
   used     = 10138264000 (9668.601989746094MB)
   free     = 11873443392 (11323.398010253906MB)
   46.05850795420205% used
G1 Young Generation:
Eden Space:
   regions  = 993
   capacity = 13824425984 (13184.0MB)
   used     = 8329887744 (7944.0MB)
   free     = 5494538240 (5240.0MB)
   60.254854368932044% used
Survivor Space:
   regions  = 5
   capacity = 41943040 (40.0MB)
   used     = 41943040 (40.0MB)
   free     = 0 (0.0MB)
   100.0% used
G1 Old Generation:
   regions  = 221
   capacity = 8145338368 (7768.0MB)
   used     = 1766433216 (1684.6019897460938MB)
   free     = 6378905152 (6083.398010253906MB)
   21.68643138190131% used

31159 interned Strings occupying 6412664 bytes.
```

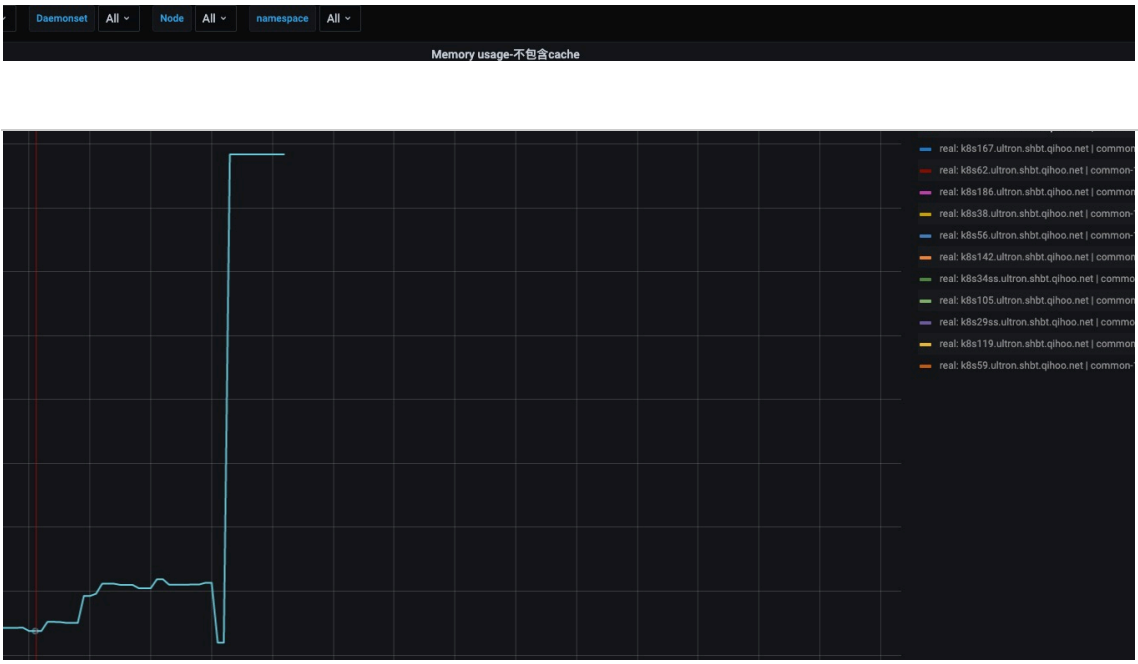
```
1159 interned Strings occupying 6412664 bytes.
flink@common-1013-2c-8s-24g-b-taskmanager-1-200 /]$ jstat -gc 1
S0C    S1C    S0U    S1U      EC       EU        OC         OU       MC     MU    CCSC   CCSU   YGC     YGCT    FGC    FGCT     GCT
0.0   32768.0  0.0   32768.0 13508608.0 1032192.0 7954432.0  1725032.4  94080.0 88466.4 12416.0 11180.1   5314  876.232   2     16.603  892.835
```

此时再次使用jmap -histo:live 1查看内存中信息，发现guava cache明显降低由570w将为170w的量级，实际存储key-value 【EstimateSpendFimension，EstimateSpendValueWithTime】对象实例对应降低。

```
 num     #instances         #bytes  class name
----------------------------------------------
   1:         18641      217755360  [B
   2:       1690945      108220480  org.apache.flink.shaded.guava18.com.google.common.cache.LocalCache$StrongAccessWriteEntry
   3:       1840559       73622360  com.mediav.overstep.spend.control.EstimateSpendDimension
   4:        109434       41019312  [Ljava.lang.Object;
   5:       1690945       40582680  com.mediav.spend.beforehand.operator.EstimateSpendValueWithTime
   6:        778533       31141320  java.math.BigDecimal
   7:       1690945       27055120  org.apache.flink.shaded.guava18.com.google.common.cache.LocalCache$StrongValueReference
   8:        258547       26364120  [C
   9:        778510       18684240  scala.math.BigDecimal
  10:        163525        6541000  java.math.BigInteger
  11:        194128        6212096  java.util.HashMap$Node
  12:        258158        6195792  java.lang.String
  13:        150331        6013240  java.util.LinkedHashMap$Entry
  14:        169120        4536968  [I
  15:        141714        4534848  java.util.concurrent.ConcurrentHashMap$Node
  16:         87213        4415144  [Ljava.util.HashMap$Node;
  17:         75819        4245864  java.util.LinkedHashMap
```

下图为GC后内存变化dashboard

但于此同时，有TM在fullGC是并不能够成功，比如这个TM并不能来得及GC就oom导致被容器kill



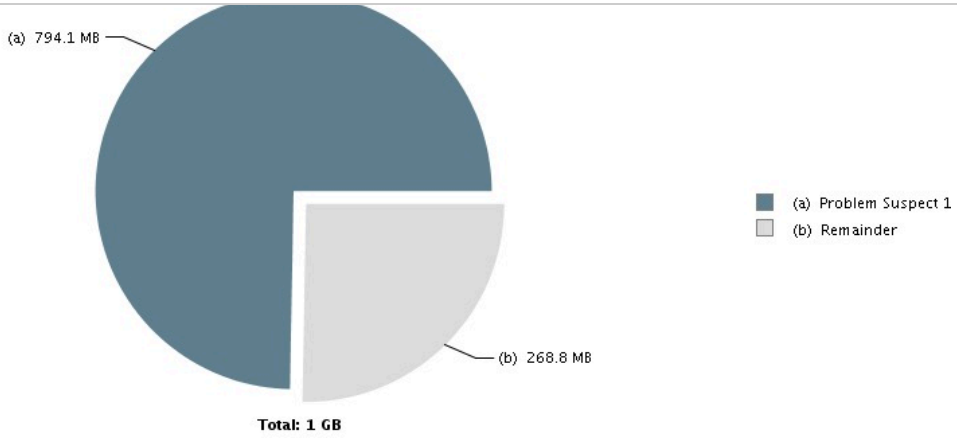
dump

由于线上出问题的Server已经被kill，此时没有到触发OOM的临界点而已，于是用jmap -histo pid | jmap -histo:live 1了JVM的堆情况，使用如下命令dump:

```
jmap -dump:format=b, file=heap.bin [pid]
```

可以发现该情况符合观察结果，ConnectedDataHandler就是引入guava cache的类名，该部分占用了大量内存，可能出现OOM。

Leaks



Problem Suspect 1

8 instances of **"com.mediav.spend.beforehand.operator.ConnectedDataHandler"**, loaded by **"org.apache.flink.util.ChildFirstClassLoader @ 0x2e0585900"** occupy **832,633,560 (74.71%)** bytes.

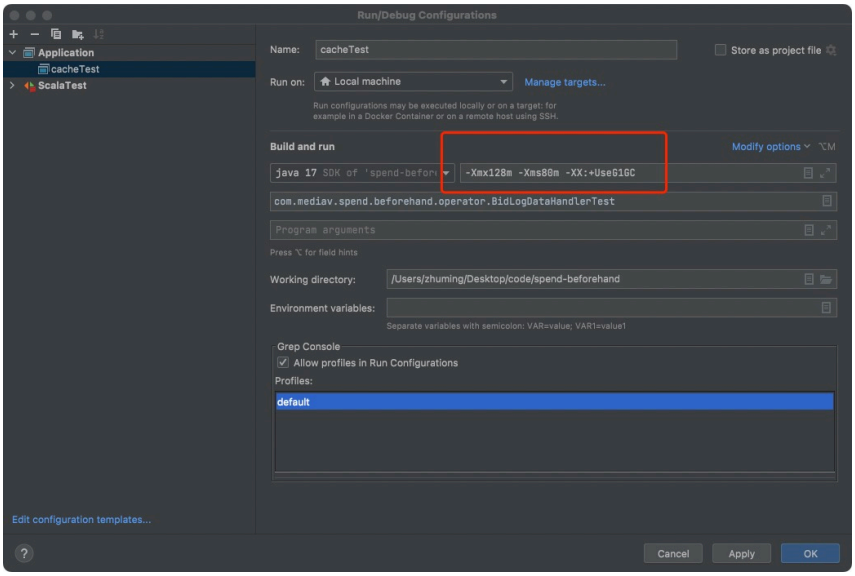
Biggest instances:

- com.mediav.spend.beforehand.operator.ConnectedDataHandler @ 0x2d5d65dc8 - 104,905,584 (9.41%) bytes.
- com.mediav.spend.beforehand.operator.ConnectedDataHandler @ 0x2d5ec89f8 - 104,892,664 (9.41%) bytes.
- com.mediav.spend.beforehand.operator.ConnectedDataHandler @ 0x2d5d90ff0 - 104,878,424 (9.41%) bytes.
- com.mediav.spend.beforehand.operator.ConnectedDataHandler @ 0x2d5e4ac50 - 104,877,688 (9.41%) bytes.
- com.mediav.spend.beforehand.operator.ConnectedDataHandler @ 0x2eab303e0 - 104,875,904 (9.41%) bytes.
- com.mediav.spend.beforehand.operator.ConnectedDataHandler @ 0x2d5dc7690 - 104,827,736 (9.41%) bytes.
- com.mediav.spend.beforehand.operator.ConnectedDataHandler @ 0x2e8027b98 - 101,707,560 (9.13%) bytes.
- com.mediav.spend.beforehand.operator.ConnectedDataHandler @ 0x2eb1c3c38 - 101,668,000 (9.12%) bytes.

Keywords
org.apache.flink.util.ChildFirstClassLoader @ 0x2e0585900
com.mediav.spend.beforehand.operator.ConnectedDataHandler
Details »

三、总结

本地guava测试，下面为jvm参数设置，-Xmx128m -Xms80m -XX:+UseG1GC

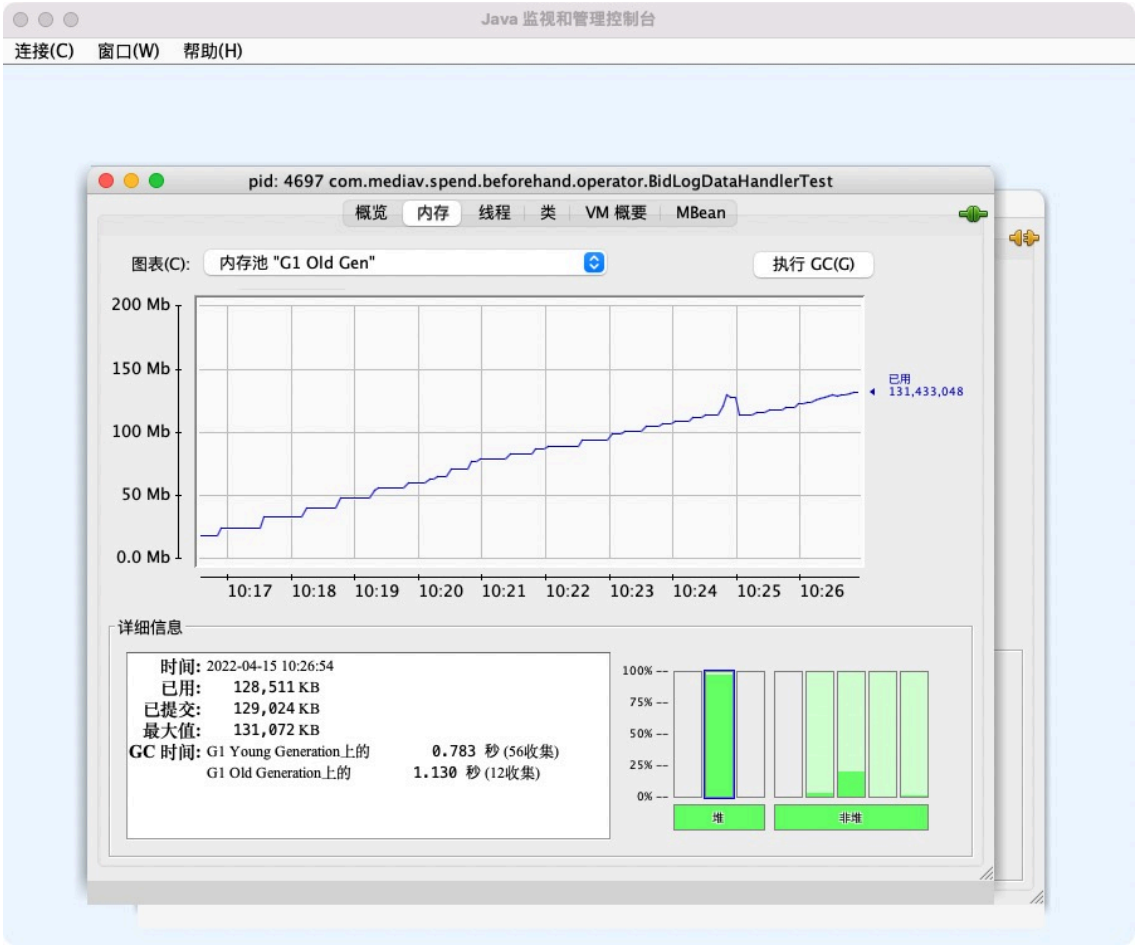


下图为测试代码，不断往guava中写入数据，其中过期时间设置为48小时，模拟无法主动释放cache

```
object BidLogDataHandlerTest  {
  def main(args: Array[String]): Unit = {
```

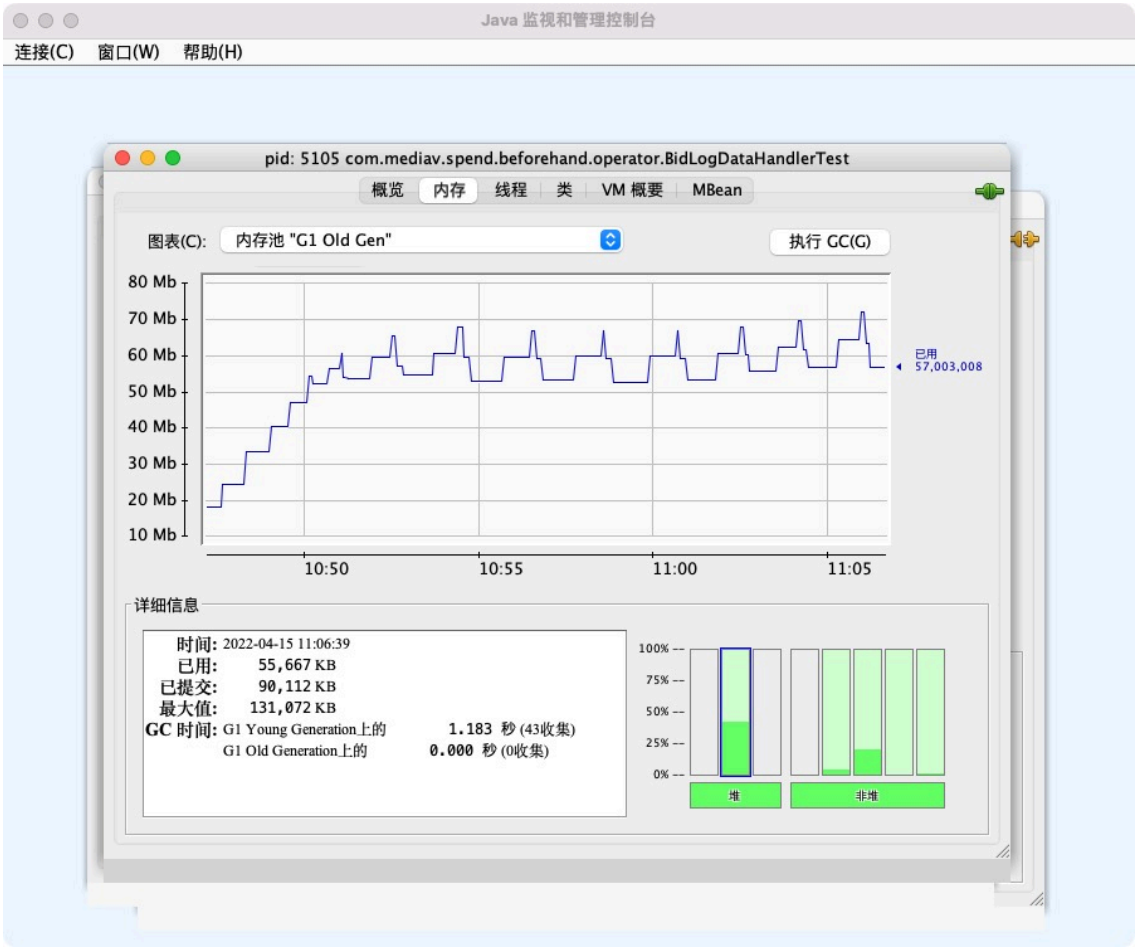
```
    .initialCapacity( initialCapacity = 1000)
    .maximumSize( size = 5000000).build[EstimateSpendDimension, EstimateSpendValueWithTime]()
  print(aerospikeCache.size())

  var i = 0
  while (true) {
    Thread.sleep( millis = 1)
    i = i + 1
    val estimateSpendEntity = new EstimateSpendEntity()
    estimateSpendEntity.setDimension(new EstimateSpendDimension().setAdspaceId((new util.Random).nextInt(10000000)))
    estimateSpendEntity.setValue(new EstimateSpendValue().setSpendEveryBid(2))
    estimateSpendEntity.setTimestamp((new util.Random).nextInt(10020000))
    val data =  EstimateSpendValueWithTime((new util.Random).nextInt(10230000), 12)
    aerospikeCache.put(estimateSpendEntity.getDimension, data)
    if (i % 1000 ==0){
      print(aerospikeCache.size() + "\n")
    }

  }
 }
}
```

随着old Gen的内存不断飙升，G1也在不断尝试fullGC，但因为guava都是强引用，并没有办法GC掉



直到最后在达到上限后，OOM

如果淘汰时间设为2分钟 expireAfterWrite(120, TimeUnit.SECONDS)，可以看到并没有触发fullGC，cache按照过期时间被清楚



疑问

为何在线上环境，一次fullGC后guava cache能够被释放掉，而且guava cache默认为强引用

可能原因

guava cache的定时删除策略有可能并不能按照设定时间删除，在cpu资源不够的时候，可能没办法删除，导致数据积压。

在老年代数据积压到一定程度，触发fullGC，此时guava cache会感知到，也会单独开线程进行数据删除；这样cpu需要同时有GC线程和cache删除线程，资源紧张导致无法完成GC和删除操作，导致内存溢出，被kill。

解决办法

定时删除cache

```
def scheduleCleanCache() : Unit = {
  object ReceiveActor extends Actor {
```

```
    null
  }
}
val actorSystem = ActorSystem("actorSystem", ConfigFactory.load())
val receiveActor = actorSystem.actorOf(Props(ReceiveActor))
import ...
val interval = 24.hour
// when at 3:00:00 am, delete local cache
val delay = {
  // 3 hour, 0 minute
  val time = LocalTime.of( hour = 3, minute = 0).toSecondOfDay
  val now = LocalTime.now().toSecondOfDay
  val fullDay = 60 * 60 * 24
  val difference = time - now
  if (difference <= 0) {
    fullDay + difference
  } else {
    difference
  }
}.seconds
actorSystem.scheduler.schedule(delay, interval,
  receiveActor, message = "cleanCache")
}
```

无标签